

Contagem paralela de objetos em matriz binária

- Sistemas Operacionais – PUCRS 2026/II
- Conectividade 8 · ANSI C · Pthreads
- Pedro Marques Sangali
- Gabriel Kirchmann Kondach
- Thiago Farias dos Santos

1. Problema e estratégia

- Objeto = região de 1s conectados por arestas ou cantos (8 vizinhos)
- Referência: flood fill sequencial
- Paralelo: faixas de linhas → rotulagem local por thread → Union-Find nas fronteiras
- Desafio: objeto que atravessa faixas não pode ser contado várias vezes

2. Sequencial (referência)

- Varre a matriz; célula 1 não visitada → novo objeto + flood fill
- Pilha explícita (iterativo): sem estouro de recursão em matrizes grandes
- Vetor 'visitado' separa células já marcadas
- Valida as 5 matrizes: 3, 4, 5, 6, 7 objetos ✓

3. Decomposição e sincronização

- Matriz dividida em p faixas contíguas de linhas (p configurável: ./paralelo arquivo p)
- Thread principal faz a faixa 0; cria $p-1$ pthreads
- Cada thread escreve só na sua faixa; matriz de entrada é só leitura
- Sem escrita compartilhada → sem corrida, sem deadlock, sem mutex
- pthread_join = barreira antes da consolidação
- ThreadSanitizer: sem avisos

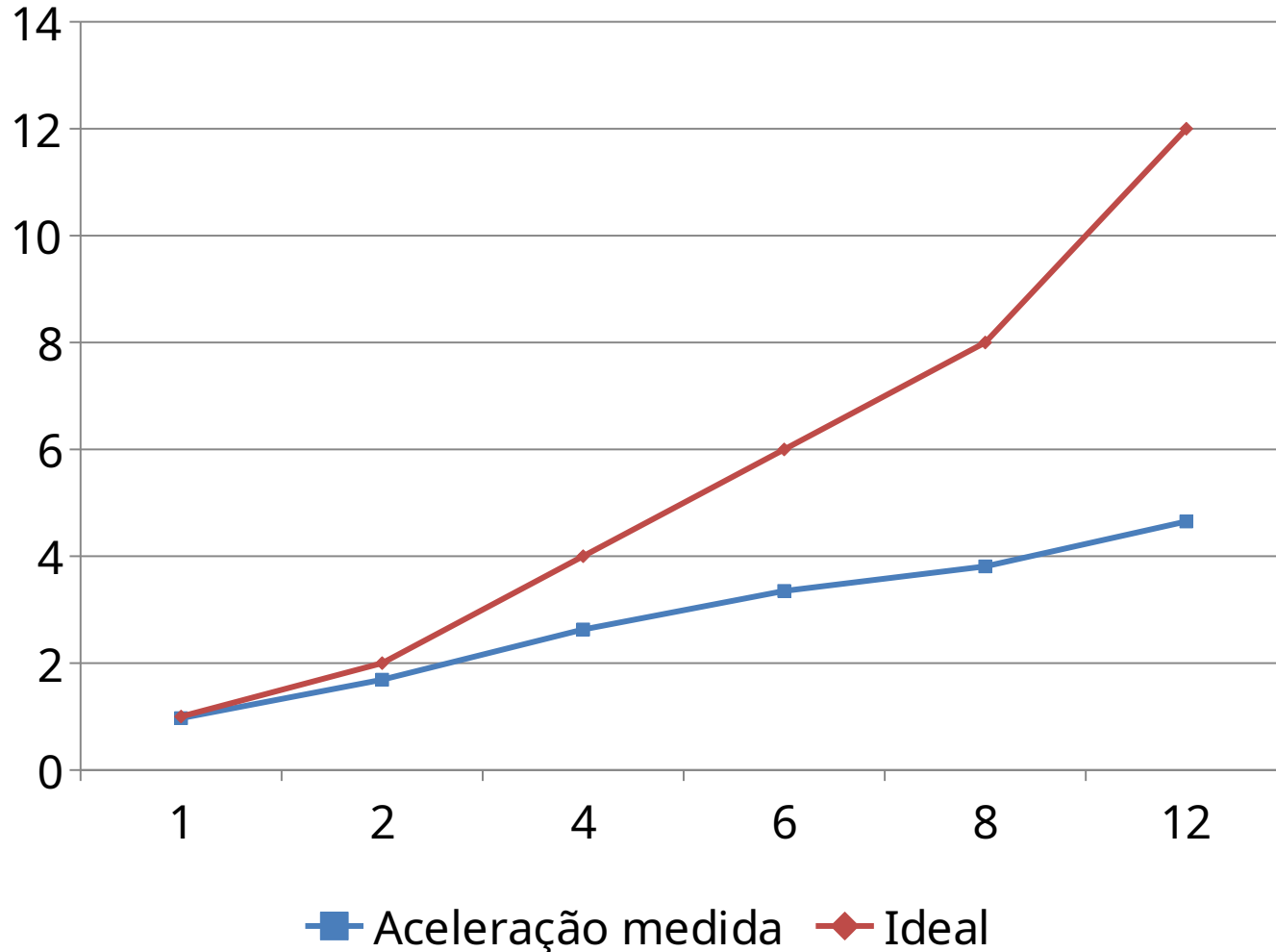
4. Consolidação nas fronteiras

- Rótulo global = rótulo local + deslocamento da faixa
- Última linha da faixa t $\times$ primeira da faixa $t+1$: vizinhos $(j-1, j, j+1)$
- Cobre vertical e diagonais; Union-Find funde rótulos equivalentes
- Objeto em 3+ faixas: união transitiva
- N° de objetos = n° de raízes do Union-Find
- Custo $O(\text{colunas})$ por fronteira

5. Demonstração e testes

- make test → matrizes obrigatórias com 2, 3, 4 e 8 threads: todas corretas
- 180 comparações aleatórias seq × paralelo: 0 divergências
- Ex. 3: diagonal e encontro de blocos · Ex. 4/5: objetos em vários blocos
- Comandos: make · make test · bin/paralelo tests/ex5.txt 4

6. Desempenho (6000×6000, mediana de 7 execuções)



- Sequencial: 772,6 ms
- 12 threads: 166,1 ms → 4,65x
- 1 thread: 0,97x (overhead sem ganho)
- Limites: Amdahl, banda de memória, SMT
- Matrizes pequenas: paralelo mais lento (custo de criar threads)

7. Conclusões

- Paralelismo efetivo: speedup até 4,65x com 12 threads
- Consolidação por Union-Find garante o mesmo resultado do sequencial
- Design sem estado compartilhado simplifica a correção
- Melhorias: blocos 2D, fila dinâmica para balancear carga, consolidação paralela